SIMATS ENGINEERING

ASSESSMENT TOOL 2 - Medium(Assignment 1)

COURSE CODE: CSA09 COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4)

Assessment Tool Weightage: 10%

QUESTIONS: 40 (each student assigned distinct questions) Total Marks: 100

ANNEXURE A

Assignment 1

Code of Conduct:

I **M.GOVARDHAN REDDY(192373028)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.GOVARDHAN REDDY

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding				
Algorithm				
Code Implementation				
Competitive Performance				
Test Case Handling				

19.Transaction records

```
import java.util.ArrayList;
```

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class FintechTransactions {

    public static void main(String[] args) {

        // HashMap to store User ID and Transaction History
        HashMap<String, List<String>> transactions = new HashMap<>();

        // Transactions for User 1
        List<String> user1 = new ArrayList<>();
        user1.add("Deposited ₹10,000");
        user1.add("Transferred ₹2,500");
        user1.add("Paid Electricity Bill ₹1,200");

        // Transactions for User 2
        List<String> user2 = new ArrayList<>();
        user2.add("Deposited ₹20,000");
        user2.add("Withdrawn ₹5,000");
        user2.add("Online Shopping ₹3,000");

        // Store in HashMap
        transactions.put("USER101", user1);
        transactions.put("USER102", user2);

        // Display Transaction History
        System.out.println("===== Transaction History =====");

        for (Map.Entry<String, List<String>> entry : transactions.entrySet()) {
```

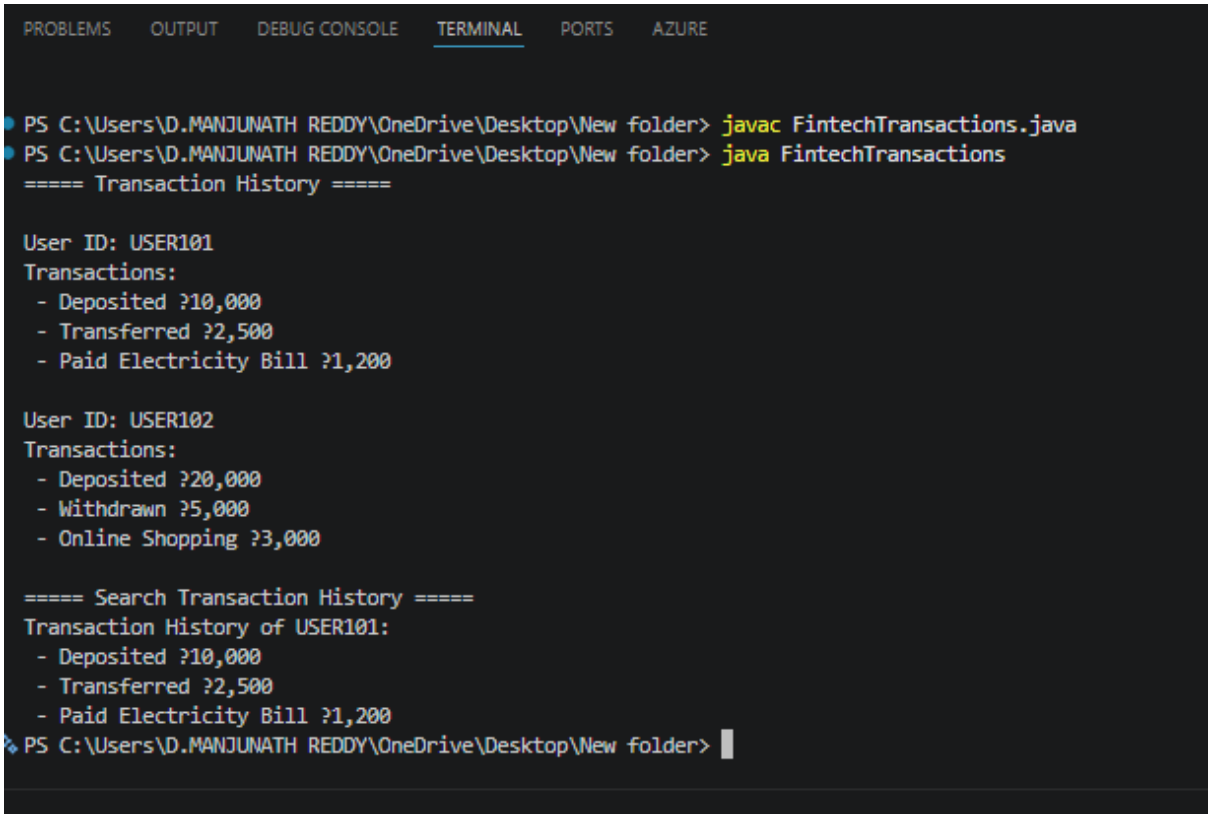
```
            System.out.println("\nUser ID: " + entry.getKey());
            System.out.println("Transactions:");

            for (String transaction : entry.getValue()) {
                System.out.println(" - " + transaction);
            }
        }

        // Display transactions of a specific user
        String searchUser = "USER101";

        System.out.println("\n===== Search Transaction History =====");

        if (transactions.containsKey(searchUser)) {
            System.out.println("Transaction History of " + searchUser + ":");
            for (String t : transactions.get(searchUser)) {
                System.out.println(" - " + t);
            }
        } else {
            System.out.println("User not found.");
        }
    }
}
```



(c) Compare Map and List

Map	**List**
Stores data as **key-value pairs**.	Stores elements in an ordered sequence.
Keys must be unique.	Duplicate elements are allowed.
Access data using a key.	Access data using an index.
Example: HashMap<String, List<String>>	Example: ArrayList<String>
Suitable for user records, bank accounts, and employee IDs.	Suitable for transaction history, shopping cart, and marks list.

(d) Secure Financial Record Management

1. Encrypt sensitive financial data before storing it.
2. Use strong authentication (passwords and multi-factor authentication).
3. Store passwords using hashing algorithms (e.g., BCrypt).
4. Apply role-based access control so only authorized users can access records.
5. Maintain audit logs for every transaction.